NOVA SCHOOL OF SCIENCE & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

MEI - Master in Computer Science and Engineering

Internet Applications Design and Implementation

Week 2 - RESTful Applications

Information

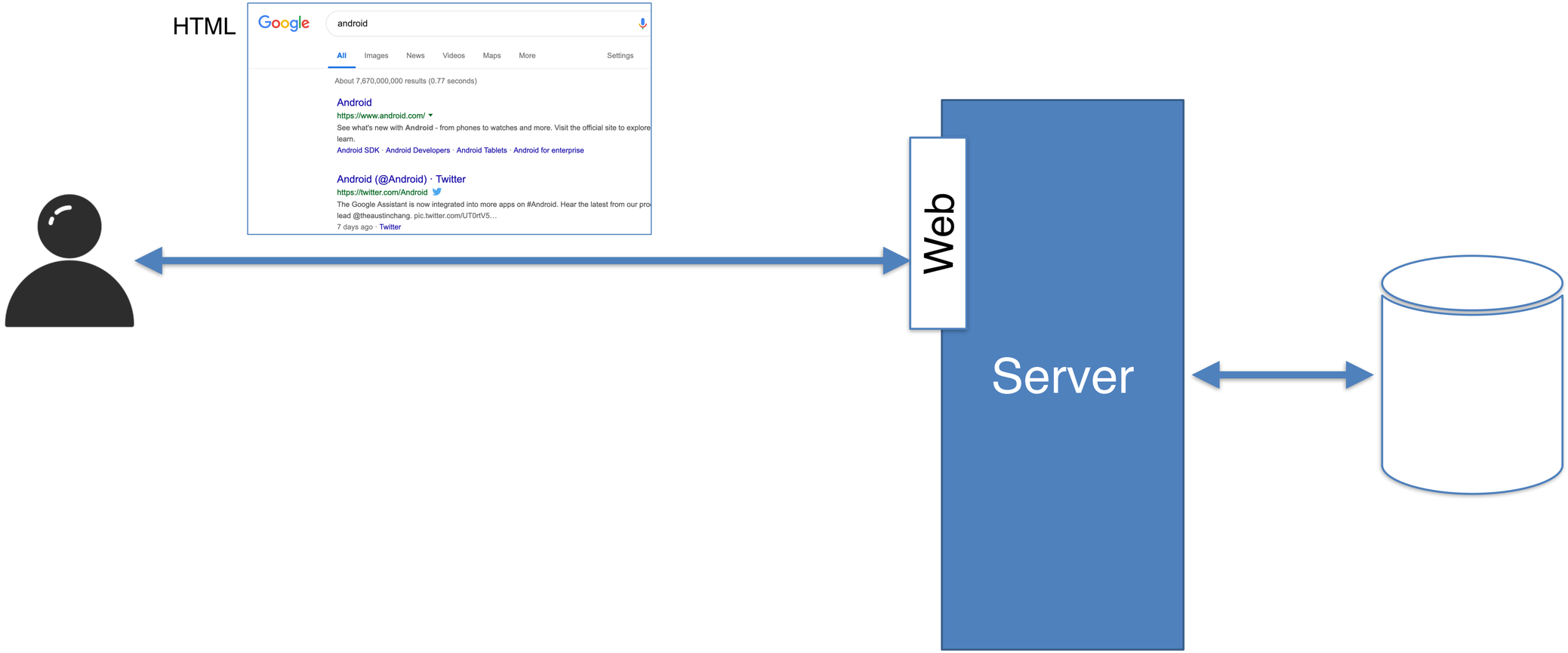
The preferred platform for asking questions about assignments is Zulip:

- Prevents duplicate questions
- Allows both teachers to respond
- Enables classmates to contribute answers
- Helps other students who may have the same question

TLDR: Please use Zulip, not email

© Pedro Alves 2026

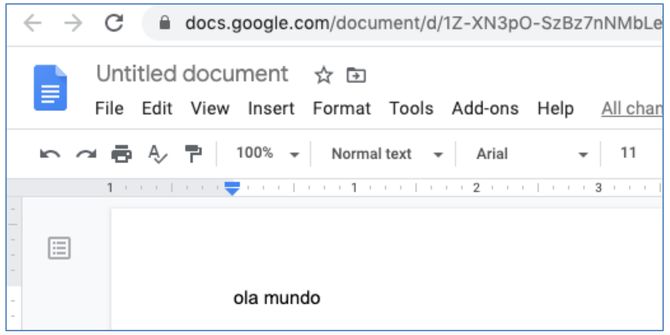
Web vs Webservices



© Pedro Alves 2026

Web vs Webservices

HTML

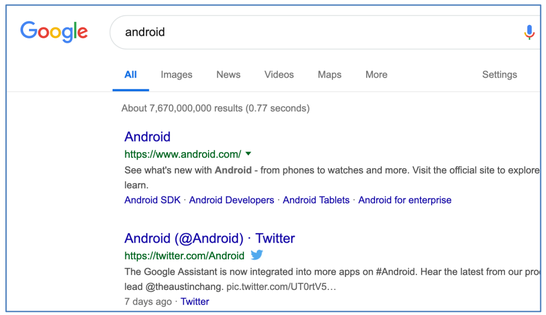


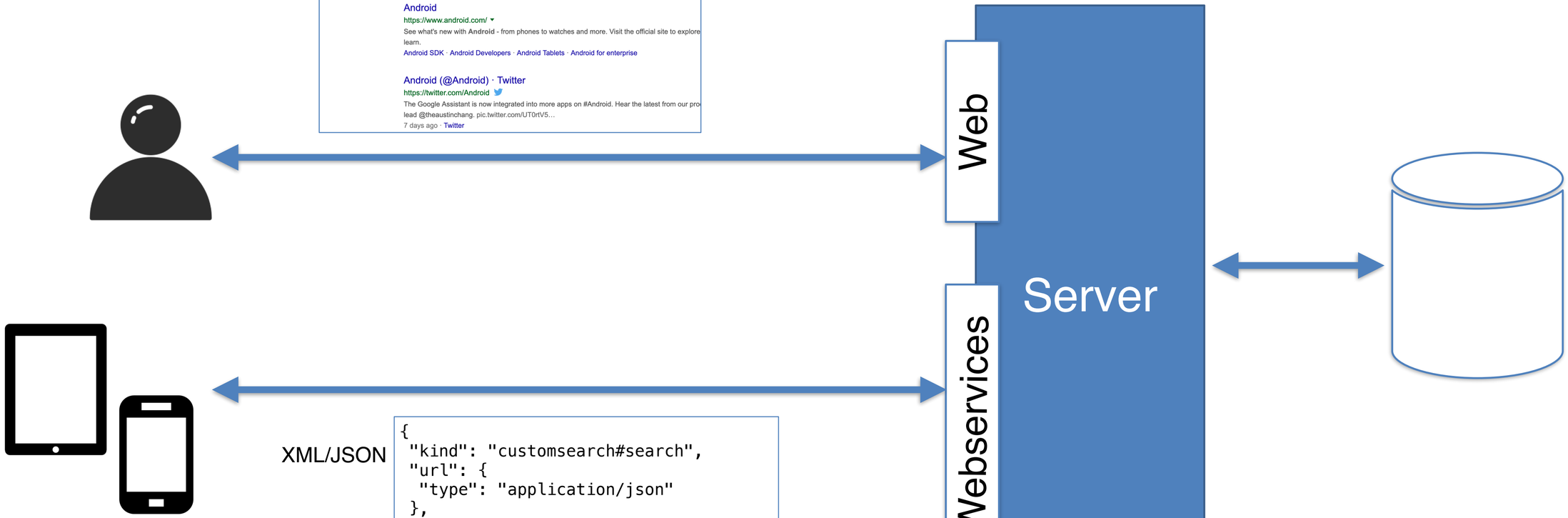
```
<div class="kix-appview-editor-container"><div class="kix-appview-editor" style="overflow-x: auto; height: 356.562px;"><div><div
class="kix-zoomdocumentplugin-outer" aria-hidden="true" style="width: 826px; height: 1066px; left: 313px; top: 4px;"><div class="kix-
zoomdocumentplugin-inner" style=""><div><div class="kix-paginateddocumentplugin"><div class="kix-paginateddocumentplugin-background docs-
ui-unprintable" style="height: 1066px; width: 816px;"></div><div><div class="kix-page docs-page kix-page-paginated"
guidedhelpid="docs_editing_area" style="background-color: rgb(255, 255, 255); width: 816px; height: 1056px;"><div class="docs-print-block
kix-page-header" style="min-height: 96px; max-height: 422.4px; width: 816px;"></div><div class="kix-page-content-wrapper"
style="background-color: rgb(255, 255, 255); height: 960px;"><div><div style="height: 864px;"><div class="kix-page-column" style="height:
864px; left: 0px; width: 816px;"><div style="margin-left: 96px; margin-right: 96px;"><div><div><div class="kix-paragraphrenderer"
id=""><div class="kix-lineview" style="height: 20px; direction: ltr; text-align: left;"><div class="kix-lineview-decorations"
style="left: 0px; top: 14.08px;"></div><div class="kix-lineview-content" style="user-select: none; margin-left: 0px; padding-top:
0px;"><span style="display: inline-block; height: 10000px"></span><span style="display: inline-block; position:relative; top:
-9985.92px;"><span class="goog-inline-block kix-lineview-text-block" style="width: 72px; padding-left: 0px;"><span class="kix-
wordhtmlgenerator-word-node" style="font-size:14.666666666666666px;font-family:Arial;color:#000000;background-color:transparent;font-
weight:400;font-style:normal;font-variant:normal;text-decoration:none;vertical-align:baseline;white-space:pre;">ola mundo<span
class="goog-inline-block" style="width:7.875px;height:17.599999999999998px"> </span></span></span></span></div></div></div></div></
div></div><div class="kix-page-column-bottom"><div class="docs-print-block" style="margin-left: 96px; margin-right: 96px;"><div
class="kix-page-column-footnotes-separator kix-page-column-footnotes-separator-hidden" style="width: 192px;"></div></div></div></div></
div></div><div class="docs-print-block kix-page-bottom" style="min-height: 96px; max-height: 422.4px; width: 816px;"></div></div><div
class="kix-page-relative-entity-container" style="top: 0px; height: 1056px;"></div></div></div><div class="docs-ui-unprintable"
style="height: 0px; background-color: rgb(255, 255, 255); display: none; width: 816px;"></div></div></div></div></div><div class="docs-
ui-unprintable"></div><div class="docs-ui-unprintable"></div><div class="docs-ui-unprintable"></div><div class="docs-ui-unprintable"></
div><div class="docs-ui-unprintable"></div><div class="kix-discussion-plugin docs-ui-unprintable" style="left: 1135px; top: 8px; text-
align: left;" dir="ltr"><div class="docos docos-stream-view docos-stream-view-legacy docos-stream-view-height-legacy docs-gm docos-
layout-anchored docos-layout-vertical" role="list"></div><div id="docs-instant-button-bubble" class="docs-material docs-instant-button-
bubble-transformation-ltr" role="button" tabindex="-1" aria-hidden="true" aria-label="Add a comment" data-tooltip="Add a comment" data-
tooltip-suspended="true" style="top: 113.8px; opacity: 0;"><div class="docs-instant-button-bubble-icon-container"><span class="docs-
instant-button-bubble-icon docos-icon-img docos-icon-img-hdpi docos-instant-icon-img docos-icon-instant-docos-ltr docos-instant-button-
bubble-icon"></span></div></div></div><div class="docs-ui-unprintable"></div></div><div class="kix-cursor docs-ui-unprintable"
style="opacity: 0; left: 414px; top: 105px;"><div class="kix-cursor-caret" style="border-color: rgb(0, 0, 0); height: 17.6px;"></div><div
class="kix-cursor-top" style="opacity: 0; display: none;"></div><div class="kix-cursor-name" style="opacity: 0; display: none;"></div></
div><div class="kix-scrollbararea"><div class="jfk-tooltip kix-scrollbarwidget" role="tooltip" aria-live="polite" style="display: none;
right: 0px; top: 0px;"><div class="jfk-tooltip-contentId" style="direction: ltr;"></div><div class="jfk-tooltip-arrow jfk-tooltip-
arrowright" style="top: 15px;"><div class="jfk-tooltip-arrowimplbefore"></div><div class="jfk-tooltip-arrowimplafter"></div></div></
div></div><div><div class="kix-embeddedobjectdragger kix-embeddedobjectdragger-embeddedentity"></div></div><div><div class="kix-
embeddedobjectdragger kix-embeddedobjectdragger-cropmode docs-ui-unprintable" style="display: none;"></div><div class="kix-
embeddedobjectdragger-cropwrapper" style="display: none;"></div></div><div></div></div></div>
```


© Pedro Alves 2026

Web vs Webservices

HTML





XML/JSON

```
{
 "kind": "customsearch#search",
 "url": {
  "type": "application/json"
 },
 "items": [
  {
   "title": "Android",
   "link": “https://www.android.com/",
   …
}
```

© Pedro Alves 2026

Web vs Webservices

HTML

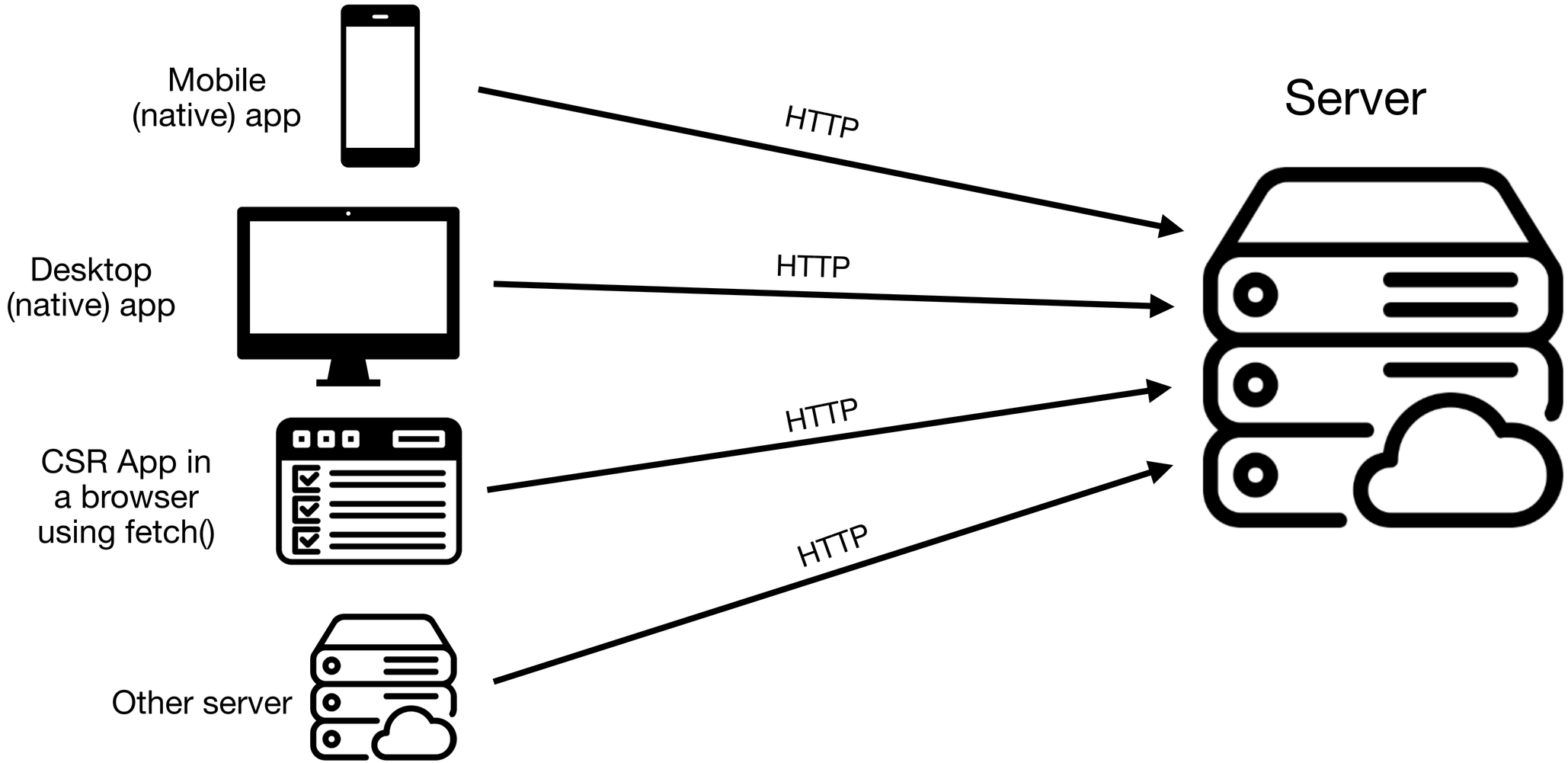

```
<div class="kix-appview-editor-container"><div class="kix-appview-editor" style="overflow-x: auto; height:
356.562px;"><div><div class="kix-zoomdocumentplugin-outer" aria-hidden="true" style="width: 826px; height:
1066px; left: 313px; top: 4px;"><div class="kix-zoomdocumentplugin-inner" style=""><div><div class="kix-
paginateddocumentplugin"><div class="kix-paginateddocumentplugin-background docs-ui-unprintable" style="height:
1066px; width: 816px;"></div><div><div class="kix-page docs-page kix-page-paginated"
guidedhelpid="docs_editing_area" style="background-color: rgb(255, 255, 255); width: 816px; height: 1056px;"><div
class="docs-print-block kix-page-header" style="min-height: 96px; max-height: 422.4px; width: 816px;"></div><div
class="kix-page-content-wrapper" style="background-color: rgb(255, 255, 255); height: 960px;"><div><div
style="height: 864px;"><div class="kix-page-column" style="height: 864px; left: 0px; width: 816px;"><div
style="margin-left: 96px; margin-right: 96px;"><div><div><div class="kix-paragraphrenderer" id=""><div
class="kix-lineview" style="height: 20px; direction: ltr; text-align: left;"><div class="kix-lineview-
decorations" style="left: 0px; top: 14.08px;"></div><div class="kix-lineview-content" style="user-select: none;
margin-left: 0px; padding-top: 0px;"><span style="display: inline-block; height: 10000px"></span><span
style="display: inline-block; position:relative; top: -9985.92px;"><span class="goog-inline-block kix-lineview-
text-block" style="width: 72px; padding-left: 0px;"><span class="kix-wordhtmlgenerator-word-node" style="font-
size:14.666666666666666px;font-family:Arial;color:#000000;background-color:transparent;font-weight:400;font-
style:normal;font-variant:normal;text-decoration:none;vertical-align:baseline;white-space:pre;">hello world<span
class="goog-inline-block" style="width:7.875px;height:17.599999999999998px"> </span></span></span></
div></div></div></div></div></div><div class="kix-page-column-bottom"><div class="docs-print-block"
style="margin-left: 96px; margin-right: 96px;"><div class="kix-page-column-footnotes-separator kix-page-column-
footnotes-separator-hidden" style="width: 192px;"></div></div></div></div></div></div><div class="docs-print-
block kix-page-bottom" style="min-height: 96px; max-height: 422.4px; width: 816px;"></div></div><div class="kix-
page-relative-entity-container" style="top: 0px; height: 1056px;"></div></div></div><div class="docs-ui-
unprintable" style="height: 0px; background-color: rgb(255, 255, 255); display: none; width: 816px;"></div></
div></div></div></div><div class="docs-ui-unprintable"></div><div class="docs-ui-unprintable"></div><div
class="docs-ui-unprintable"></div><div class="docs-ui-unprintable"></div><div class="docs-ui-unprintable"></
div><div class="kix-discussion-plugin docs-ui-unprintable" style="left: 1135px; top: 8px; text-align: left;"
dir="ltr"><div class="docos docos-stream-view docos-stream-view-legacy docos-stream-view-height-legacy docs-gm
docos-layout-anchored docos-layout-vertical" role="list"></div><div id="docs-instant-button-bubble" class="docs-
material docs-instant-button-bubble-transformation-ltr" role="button" tabindex="-1" aria-hidden="true" aria-
label="Add a comment" data-tooltip="Add a comment" data-tooltip-suspended="true" style="top: 113.8px; opacity:
0;"><div class="docs-instant-button-bubble-icon-container"><span class="docs-instant-button-bubble-icon docos-
icon-img docos-icon-img-hdpi docos-instant-icon-img docos-icon-instant-docos-ltr docos-instant-button-bubble-
icon"></span></div></div></div><div class="docs-ui-unprintable"></div></div><div class="kix-cursor docs-ui-
unprintable" style="opacity: 0; left: 414px; top: 105px;"><div class="kix-cursor-caret" style="border-color:
rgb(0, 0, 0); height: 17.6px;"></div><div class="kix-cursor-top" style="opacity: 0; display: none;"></div><div
class="kix-cursor-name" style="opacity: 0; display: none;"></div></div><div class="kix-scrollbararea"><div
class="jfk-tooltip kix-scrollbarwidget" role="tooltip" aria-live="polite" style="display: none; right: 0px; top:
0px;"><div class="jfk-tooltip-contentId" style="direction: ltr;"></div><div class="jfk-tooltip-arrow jfk-tooltip-
arrowright" style="top: 15px;"><div class="jfk-tooltip-arrowimplbefore"></div><div class="jfk-tooltip-
arrowimplafter"></div></div></div></div><div><div class="kix-embeddedobjectdragger kix-embeddedobjectdragger-
embeddedentity"></div></div><div><div class="kix-embeddedobjectdragger kix-embeddedobjectdragger-cropmode docs-
ui-unprintable" style="display: none;"></div><div class="kix-embeddedobjectdragger-cropwrapper" style="display:
none;"></div></div><div></div></div></div>
```


JSON


```
{
    "document": {
        "body": "hello world”
    }
}
```


© Pedro Alves 2026

Webservices



© Pedro Alves 2026

RESTful interface design

- Architectural style (convention)
  - Simpler, more efficient way of implementing web services
  - Built on top of basic HTTP
- Decoupling from data-centric server side applications and client user-centric applications
- Several possible implementations
  - Web-service style pure JSON/XML Data
  - Complete/partial HTML view responses
  - Javascript code responses (e.g. Rails AJAX responses)
- Fielding, Roy Thomas (2000). "Chapter 5: Representational State Transfer (REST)". Architectural Styles and the Design of Network-based Software Architectures (Ph.D.). University of California, Irvine

© Pedro Alves 2026

REST - Representational State Transfer

- Resource Based
- Representation
- Uniform Interface
- Stateless
- Cacheable
- Client-Server
- Layered System
- Code on Demand (optional, not talking about it)

© Pedro Alves 2026

REST

Resource Based

Resource Based

- vs Action Based
- Nouns and not verbs to identify data in the system
- Identified (represented) by URI
- Aliasing is admissible

© Pedro Alves 2026

Representation

Representation

- JSON or XML representation of the state of a given resource transferred between client and server at a given verb in a given URL.
- Well identified interface (the information retrieved at an URL — the type)

© Pedro Alves 2026

REST

Uniform Interface

Uniform Interface

- Standard HTTP verbs (GET, POST, PUT, DELETE, …)
- Standard HTTP response (status code, info in the response body)
- Uniform structure of URIs with a name, identifying the resource
- References inside responses must be complete.

© Pedro Alves 2026

REST

Stateless

Stateless

- Server does not hold session state
- Messages are self contained

© Pedro Alves 2026

REST

Cacheable

Cacheable

- Responses can be tagged as cacheable (in the server)
- (also) Bookmarkable

© Pedro Alves 2026

Layered System

Layered System

- Establishes an API between a client and a “database”

© Pedro Alves 2026

REST

Actions

HTTP Methods represent standardized actions

- GET = request resources
- POST = create resources
- PUT = update or create resources
- PATCH = update resources
- DELETE = delete resources

© Pedro Alves 2026

REST

Actions

Feature	POST	PUT	PATCH
Purpose	Create new resource	Create or fully replace a resource at a known URI	Partially update a resource
Request Body	Full representation	Full representation	Partial representation
Usual Response	201 Created (often with Location)	200 OK, 201 Created	200 OK
Idempotent?	No	Yes	Yes
Resource doesn’t exist	Creates	Creates	Fails
Example URL	POST /users	PUT /users/123	PATCH /users/123

© Pedro Alves 2026

REST

Response codes

HTTP Response codes represent operation results

- 200 OK, 201 Created, 204 No Content
- 3xx Redirection (not modified)
- 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- 5xx Server Error

4xx - Client-side problem (no point in retrying, have to change request)

5xx - Server-side problem (maybe useful to retry, wait, contact admin?)

© Pedro Alves 2026

REST

Query params

Use URL query params for:

- searching
- filtering
- sorting
- pagination

https://api.github.com/repos/octocat/Spoon-Knife/issues?state=open&sort=updated&per_page=20&page=2

© Pedro Alves 2026

REST

Example

Application to manage contacts of partner companies (e.g. for security clearance in events)

- **Resources**
  - Companies (name, address, email, list of contacts (employees))
  - Contact/Employee (name, email, job, company)
- **Operations (CRUD)**
  - List, add, update, and delete resources

© Pedro Alves 2026

REST

Example

Partner companies

- `GET /companies` - List all the companies
- `GET /companies?search=<criteria>` - List all the companies that contain the substring <criteria>
- `POST /companies` - Create a company described in the payload. The request body must include all the necessary attributes.
- `GET /companies/{id}` - Shows the company with identifier {id}
- `PATCH /companies/{id}` - Updates the company with {id} having values in the payload. The updatable items may vary (name, email, etc.)
- `PUT /companies/{id}` - Replaces all the attributes of company with {id}
- `DELETE /companies/{id}` - Removes the company with {id}

© Pedro Alves 2026

REST

Example

Partner contacts

- `GET /contacts` - List all the contacts
- `GET /contacts?search=<criteria>` - List all the contacts that contain the substring <criteria>
- `POST /contacts` - Create a contact described in the payload. The request body must include all the necessary attributes.
- `GET /contacts/{id}` - Shows the contact with identifier {id}
- `PATCH /contacts/{id}` - Updates the contact with {id} having values in the payload. The updatable items may vary (name, email, etc.)
- `PUT /contacts/{id}` - Replaces all the attributes of contact with {id}
- `DELETE /contacts/{id}` - Removes the contact with {id}

© Pedro Alves 2026

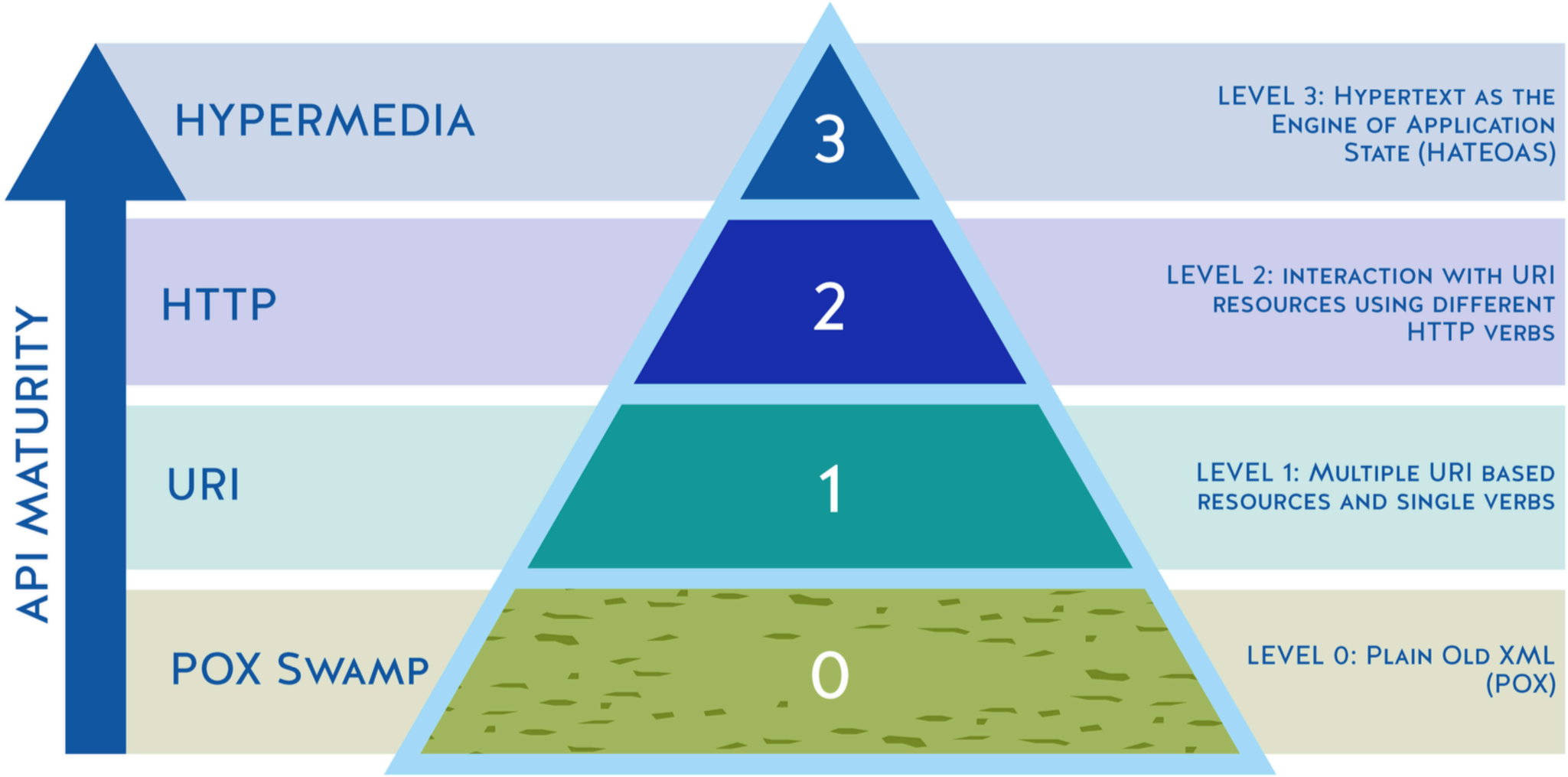
Example

Partner contacts of companies

- `GET /companies/{id}/contacts` - List all the contacts of a company
- `GET /companies/{id}/contacts?search=<criteria>` - List all the contacts of a company that contain the substring <criteria>
- `POST /companies/{id}/contacts` - Create a contact of company {id} described in the payload. The request body must include all the necessary attributes.
- `GET /companies/{id}/contacts/{cid}` – Shows the contact of company {id} with identifier {cid}

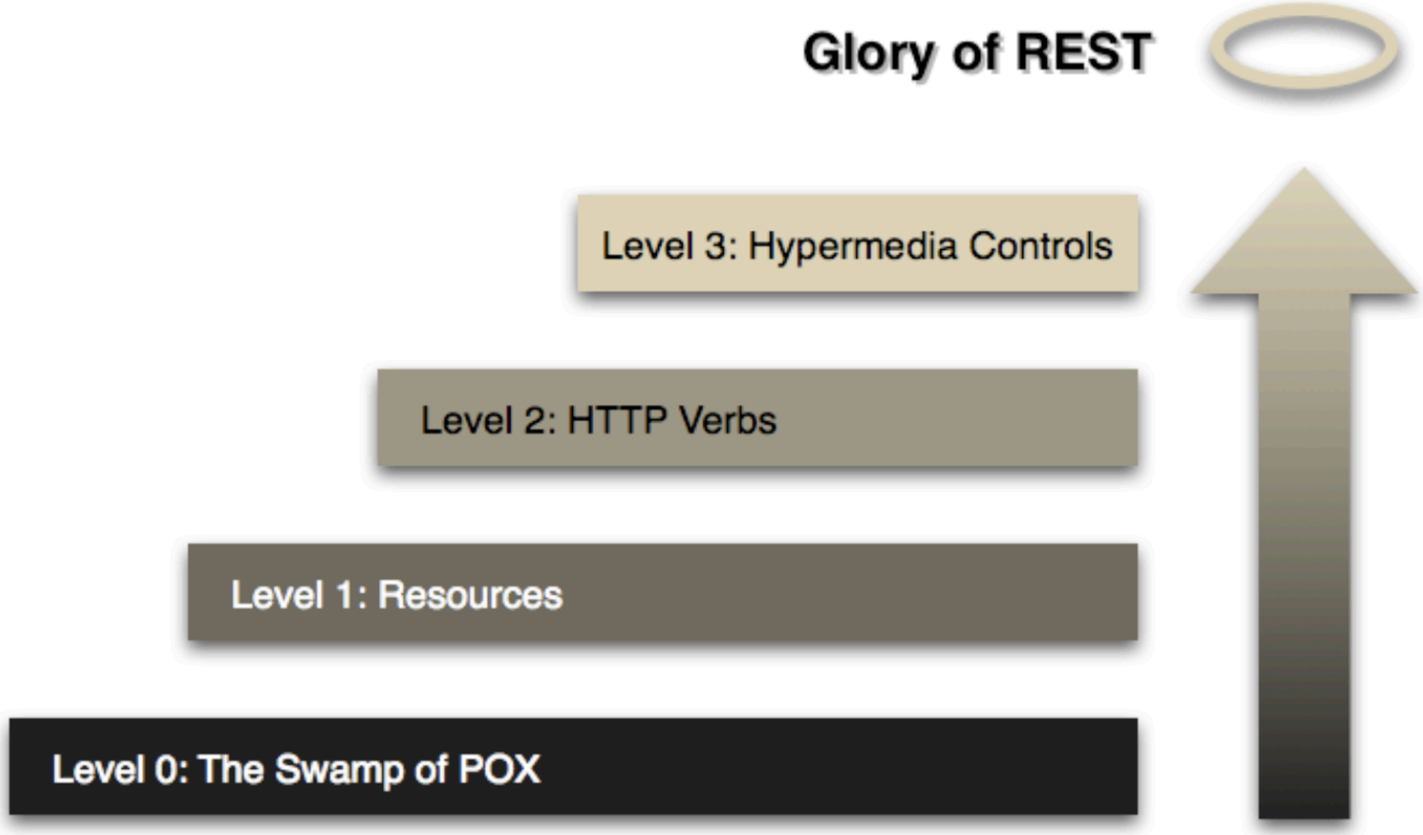
© Pedro Alves 2026

Richardson Maturity Model



© Pedro Alves 2026

Richardson Maturity Model



https://martinfowler.com/articles/richardsonMaturityModel.html

https://restcookbook.com/Miscellaneous/richardsonmaturitymodel/

© Pedro Alves 2026

Richardson Maturity Model - Level 0

POX Swamp

- To send an XML/JSON that contains everything: operation, arguments, options

```
POST /appointmentService HTTP/1.1
[various other headers]

<openSlotRequest date = "2010-01-04" doctor = "mjones"/>
```

```
HTTP/1.1 200 OK
[various headers]

<openSlotList>
  <slot start = "1400" end = "1450">
    <doctor id = "mjones"/>
  </slot>
  <slot start = "1600" end = "1650">
    <doctor id = "mjones"/>
  </slot>
</openSlotList>
```

© Pedro Alves 2026

Richardson Maturity Model - Level 0

POX Swamp

- To send an XML/JSON that contains everything: operation, arguments, options

```
POST /appointmentService HTTP/1.1
[various other headers]

<appointmentRequest>
  <slot doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
</appointmentRequest>
```

```
HTTP/1.1 200 OK
[various headers]

<appointmentRequestFailure>
  <slot doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
  <reason>Slot not available</reason>
</appointmentRequestFailure>
```

© Pedro Alves 2026

Richardson Maturity Model - Level 0

POX Swamp

- To send an XML/JSON that contains everything: operation, arguments, options

```
POST /appointmentService HTTP/1.1
[various other headers]

<appointmentRequest>
  <slot doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
</appointmentRequest>
```

```
HTTP/1.1 200 OK
[various headers]

<appointmentRequestFailure>
  <slot doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
  <reason>Slot not available</reason>
</appointmentRequestFailure>
```

Similar to imperative programming - call a function passing arguments

© Pedro Alves 2026

Richardson Maturity Model - Level 1

Multiple URI Based Resources and Single verbs

```
POST /doctors/mjones HTTP/1.1
[various other headers]

<openSlotRequest date = "2010-01-04"/>
```

```
HTTP/1.1 200 OK
[various headers]

<openSlotList>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1650"/>
</openSlotList>
```

```
POST /slots/1234 HTTP/1.1
[various other headers]

<appointmentRequest>
  <patient id = "jsmith"/>
</appointmentRequest>
```

```
HTTP/1.1 200 OK
[various headers]

<appointment>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
</appointment>
```

Similar to object-oriented programming - call a method on a given object

© Pedro Alves 2026

Richardson Maturity Model - Level 2

Interaction with URI resources using different HTTP verbs

```
GET /doctors/mjones/slots?date=20100104&status=open HTTP/1.1
Host: royalhope.nhs.uk
```

```
HTTP/1.1 200 OK
[various headers]

<openSlotList>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1650"/>
</openSlotList>
```

© Pedro Alves 2026

Richardson Maturity Model - Level 2

Interaction with URI resources using different HTTP verbs

```
POST /slots/1234 HTTP/1.1
[various other headers]

<appointmentRequest>
  <patient id = "jsmith"/>
</appointmentRequest>
```

```
HTTP/1.1 201 Created
Location: slots/1234/appointment
[various headers]

<appointment>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>
  <patient id = "jsmith"/>
</appointment>
```

```
HTTP/1.1 409 Conflict
[various headers]

<openSlotList>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1650"/>
</openSlotList>
```

Richardson Maturity Model - Level 3

Hypermedia Controls - HATEOAS

Resources are interconnected by links in the response, one entry point

```
GET /doctors/mjones/slots?date=20100104&status=open HTTP/1.1
Host: royalhope.nhs.uk
```

```
HTTP/1.1 200 OK
[various headers]

<openSlotList>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450">
    <link rel = "/linkrels/slot/book"
          uri = "/slots/1234"/>
  </slot>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1650">
    <link rel = "/linkrels/slot/book"
          uri = "/slots/5678"/>
  </slot>
</openSlotList>
```

© Pedro Alves 2026

Exercise

Spot the REST violations (level 2 of Richardson Maturity Model).
Tell what should be used instead.

```
1. GET /api/getStudents
2. POST /api/students/delete?id=123
3. GET /api/student/123/grades/calculate
4. GET /api/courses/7/students
5. PUT /api/students
6. GET /api/courses?courseId=7
7. POST /api/courses/7/students/123
8. GET /api/student/123
9. PATCH /api/students/123?name=Alice&email=a@x.com
```

© Pedro Alves 2026

Solution

1. `GET /api/getStudents` -> GET /api/students
2. `POST /api/students/delete?id=123` -> DELETE /api/students/123
3. `GET /api/student/123/grades/calculate` -> calculate is a verb
4. `GET /api/courses/7/students` ✅
5. `PUT /api/students` -> PUT /api/students/{id}
6. `GET /api/courses?courseId=7&sort=name` -> GET /api/courses/7?sort=name
7. `POST /api/courses/7/students/123` -> POST /api/courses/7/students
8. `GET /api/student/123` -> GET /api/students/123
9. `PATCH /api/students/123?name=Alice&email=a@x.com`
   -> PATCH /api/students/123 (the fields go into the payload)

© Pedro Alves 2026

REST

Real REST API examples

- Google Glasses API - https://developers.google.com/glass/v1/reference
- Tesla API - https://www.teslaapi.io/
- GitHub API - https://docs.github.com/en/rest

© Pedro Alves 2026

REST

Google Glasses API - https://developers.google.com/glass/v1/reference

Contacts

For Contacts Resource details, see the resource representation page.

Method	HTTP request	Description
URIs relative to https://www.googleapis.com/mirror/v1, unless otherwise noted		
delete	`DELETE /contacts/id`	Deletes a contact.
get	`GET /contacts/id`	Gets a single contact by ID.
insert	`POST /contacts`	Inserts a new contact.
list	`GET /contacts`	Retrieves a list of contacts for the authenticated user.
patch	`PATCH /contacts/id`	Updates a contact in place. This method supports patch semantics.
update	`PUT /contacts/id`	Updates a contact in place.

© Pedro Alves 2026

REST

Google Glasses API - https://developers.google.com/glass/v1/reference

Timeline.attachments

For Timeline.attachments Resource details, see the resource representation page.

Method	HTTP request	Description
URIs relative to https://www.googleapis.com/mirror/v1, unless otherwise noted		
delete	`DELETE /timeline/itemId/attachments/attachmentId`	Deletes an attachment from a timeline item.
get	`GET /timeline/itemId/attachments/attachmentId`	Retrieves an attachment on a timeline item by item ID and attachment ID.
insert	`POST https://www.googleapis.com/upload/mirror/v1/timeline/itemId/attachments`	Adds a new attachment to a timeline item.
list	`GET /timeline/itemId/attachments`	Returns a list of attachments for a timeline item.

© Pedro Alves 2026

REST

Honk Horn

`POST` `https://owner-api.teslamotors.com/api/1/vehicles/:id/command/honk_horn`

Honks the horn of the vehicle once

Path Parameters

Name	Type	Description
:id	integer	The vehicle `{id}` from the vehicle list

Headers

Name	Type	Description
Authorization	string	Bearer `{access_token}` from authentication

200

```
{
  "response": {
    "result": true,
    "reason": ""
  }
}
```

Tesla API - https://www.teslaapi.io/vehicles/commands#honk-horn

© Pedro Alves 2026

REST

GitHub API - https://docs.github.com/en/rest/commits/commits?apiVersion=2022-11-28

List commits

GET /repos/{owner}/{repo}/commits

cURL | JavaScript | GitHub CLI

```
curl -L \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer <YOUR-TOKEN>" \
  -H "X-GitHub-Api-Version: 2022-11-28" \
  https://api.github.com/repos/OWNER/REPO/commits
```

© Pedro Alves 2026

Documenting REST APIs

History

- 2010 - Tony Tam creates Swagger, an API description format, while working at Wordnik.
- 2015 - The Swagger project/specification was acquired by SmartBear Software.
- Later in 2015 - SmartBear donated the Swagger Specification to the newly formed OpenAPI Initiative under the Linux Foundation to ensure vendor-neutral governance.
- 2016 - The Swagger Specification was renamed to the OpenAPI Specification (OAS).
- 2017 - OpenAPI 3.0.0 (a major redesign from Swagger/OpenAPI 2.0).
- 2021 - OpenAPI 3.1.0 further modernized the spec by aligning more closely with JSON Schema

© Pedro Alves 2026

OpenAPI

- Specification language for REST APIs (Yaml or JSON)
- Provides online (reflective) information on service(s)
  - Paths and operations (GET /companies, POST /employees)
  - Input and output parameters for each operation (samples)
  - Authentication methods
- Contact information, license, terms of use and other information.
- Design, implementation and validation tools
- Editor, UI, Codegen, Spring Annotations
- Extensions to include more information about contracts

© Pedro Alves 2026

OpenAPI

Yaml Specification

General information about the API

```yaml
openapi: 3.1.0
info:
  title: Example Company API
  version: 1.0.0
  summary: Simple demo API in OpenAPI 3.1
  description: |
    Demonstrates paths, parameters, request bodies, responses, and security.
  termsOfService: https://example.com/terms
  contact:
    name: API Support
    email: support@example.com
  license:
    name: MIT
    identifier: MIT

servers:
  - url: https://api.example.com/v1
    description: Production

security:
  - bearerAuth: []
```

© Pedro Alves 2026

OpenAPI

Yaml Specification

Specific information about each operation

```yaml
paths:
  /companies:
    get:
      operationId: listCompanies
      summary: List companies
      parameters:
        - name: limit
          in: query
          description: Max number of companies to return
          required: false
          schema:
            type: integer
      responses:
        "200":
          description: A list of companies
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/Company"
              examples:
                sample:
                  value:
                    - id: "c_123"
                      name: "Acme Corp"
                    - id: "c_456"
                      name: "Globex"
```

© Pedro Alves 2026

OpenAPI

Yaml Specification

Information about authentication

```yaml
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

© Pedro Alves 2026

OpenAPI

Yaml Specification

Information about error responses

```yaml
responses:
    BadRequest:
      description: Invalid input
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/Error"
          examples:
            badInput:
              value:
                code: "bad_request"
                message: "Invalid request payload."
```

© Pedro Alves 2026

OpenAPI

Yaml Specification

Information about schemas/data types

```yaml
schemas:
    # OAS 3.1 uses JSON Schema vocab
    Company:
      type: object
      additionalProperties: false
      required: [id, name]
      properties:
        id:
          type: string
          description: Company identifier
        name:
          type: string
          minLength: 1
```

https://json-schema.org/specification

© Pedro Alves 2026

OpenAPI

Spring Boot

Spring Boot 3 automatically generates a spec for all endpoints (OpenAPI 3.1 format) during application startup

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.15</version>
</dependency>
```

© Pedro Alves 2026

OpenAPI
Spring Boot

```
@Tag(name = "Bookstore API", description = "the bookstore API")
interface BookstoreAPI {

    @Operation(
        summary = "Add a new book to the store",
        operationId = "addBook",
        tags = ["book"]
    )
    @ApiResponses(
        value = [
            ApiResponse(
                responseCode = "201",
                description = "Book created successfully",
                headers = [Header(
                    name = "Location",
                    description = "URI of the newly created book",
                    schema = Schema(type = "string", format = "uri")
                )]
            ),
            ApiResponse(responseCode = "400", description = "Invalid input")
        ]
    )
    @RequestMapping(
        value = ["/books"],
        produces = ["application/json"],
        consumes = ["application/json"],
        method = [RequestMethod.POST]
    )
    fun addBook(
        @Parameter(
            description = "Book object that needs to be added to the store",
            required = true
        ) @Valid @RequestBody book: Book
    ): ResponseEntity<Void>
```

© Pedro Alves 2026

Swagger UI

localhost:8080/swagger-ui/index.html#/bookstore/addBook

OpenAPI
Spring Boot

POST **/books** Add a new book to the store

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "id": 0,
  "title": "string",
  "author": "string",
  "isbn": "string",
  "price": 0.1
}
```

Responses

Code	Description	Links
201	Book created successfully **Headers:** **Name** / **Description** / **Type** `Location` / `URI of the newly created book` / `string`	*No links*
400	Invalid input	*No links*

© Pedro Alves 2026

OpenAPI

Spring Boot

```
data class Book(
    val id: Long,
    val title: String,
    val author: String,
    @field:Schema(description = "ISBN must have 13 digits", pattern = "^\\d{13}$")
    val isbn: String,
    @field:Schema(description = "Price must be greater than zero", exclusiveMinimum = true, minimum = "0")
    val price: Double,
)
```

Schemas

Book ^ Collapse all `object`

Book object that needs to be added to the store

- `id*` `integer` `int64`
- `title*` `string`
- `author*` `string`
- `isbn*` ^ Collapse all `string` `matches ^\d{13}$`
  ISBN must have 13 digits
- `price*` ^ Collapse all `number` `≥ 0` `double`
  Price must be greater than zero

© Pedro Alves 2026

OpenAPI

Spring Boot

```
@RestController
class BookstoreController : BookstoreAPI {

    override fun addBook(book: Book): ResponseEntity<Void> {

        // TODO: implement

        val location = ServletUriComponentsBuilder
            .fromCurrentRequest()
            .path("/{id}")
            .buildAndExpand(book.id)
            .toUri()

        return ResponseEntity.created(location).build()
    }

}
```

```
HTTP/1.1 201 Created
Location: https://api.example.com/books/42
```

You need to have a (minimal) RestController implementing the API

otherwise the spec won't be generated

© Pedro Alves 2026

OpenAPI
Spring Boot

http://localhost:8080/swagger-ui.html

human readable

http://localhost:8080/v3/api-docs

machine readable


```
{
  "openapi": "3.1.0",
  "info": {
    "title": "OpenAPI definition",
    "version": "v0"
  },
  "servers": [
    {
      "url": "http://localhost:8080",
      "description": "Generated server url"
    }
  ],
  "tags": [
    {
      "name": "Bookstore API",
      "description": "the bookstore API"
    }
  ],
  "paths": {
    "/books": {
      "post": {
        "tags": [
          "book",
          "Bookstore API"
        ],
        "summary": "Add a new book to the store”,
        ...
```


© Pedro Alves 2026

REST server implementation

Basic Servlet

```
GET /students
GET /students/{id}
```

```
class PagesServlet : HttpServlet() {

    private val students = listOf(
        Student(1, "Alice", 12345),
        Student(2, "Bob", 67890)
    )

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        when {
            path == "/students" || path == "/students/" -> handleGetStudents(resp)

            path.matches(Regex("^/students/\\d+$")) -> {
                val id = path.removePrefix("/students/").toLong()
                handleGetStudent(id, resp)
            }
        }
    }

    private fun handleGetStudents(resp: HttpServletResponse) {
        val json = students.joinToString(",", "[", "]") { it.toJson() }
        resp.writer.write(json)
    }

    private fun handleGetStudent(id: Long, resp: HttpServletResponse) {
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

```
data class Student(val id: Long, val name: String, val number: Int) {
    fun toJson() = """{"id": $id, "name": "$name", "number": $number}"""
}
```

© Pedro Alves 2026

GET /students

GET /students/{id}

REST server implementation

Basic Servlet

```
class PagesServlet : HttpServlet() {

    private val students = listOf(
        Student(1, "Alice", 12345),
        Student(2, "Bob", 67890)
    )

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        when {
            path == "/students" || path == "/students/" -> handleGetStudents(resp)

            path.matches(Regex("^/students/\\d+$")) -> {
                val id = path.removePrefix("/students/").toLong()
                handleGetStudent(id, resp)
            }
        }
    }

    private fun handleGetStudents(resp: HttpServletResponse) {
        val json = students.joinToString(",", "[", "]") { it.toJson() }
        resp.writer.write(json)
    }

    private fun handleGetStudent(id: Long, resp: HttpServletResponse) {
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

```
data class Student(val id: Long, val name: String, val number: Int) {
    fun toJson() = """{"id": $id, "name": "$name", "number": $number}"""
}
```

Router

decides which component will handle this request

© Pedro Alves 2026

GET /students

GET /students/{id}

REST server implementation

Basic Servlet

```
class PagesServlet : HttpServlet() {

    private val students = listOf(
        Student(1, "Alice", 12345),
        Student(2, "Bob", 67890)
    )

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        when {
            path == "/students" || path == "/students/" -> handleGetStudents(resp)

            path.matches(Regex("^/students/\\d+$")) -> {
                val id = path.removePrefix("/students/").toLong()
                handleGetStudent(id, resp)
            }
        }
    }

    private fun handleGetStudents(resp: HttpServletResponse) {
        val json = students.joinToString(",", "[", "]") { it.toJson() }
        resp.writer.write(json)
    }

    private fun handleGetStudent(id: Long, resp: HttpServletResponse) {
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

```
data class Student(val id: Long, val name: String, val number: Int) {
    fun toJson() = """{"id": $id, "name": "$name", "number": $number}"""
}
```

Problems with this implementation?

© Pedro Alves 2026

REST server implementation

Basic Servlet

```
GET /students
GET /students/{id}
```

```
class PagesServlet : HttpServlet() {

    private val students = listOf(
        Student(1, "Alice", 12345),
        Student(2, "Bob", 67890)
    )

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        when {
            path == "/students" || path == "/students/" -> handleGetStudents(resp)

            path.matches(Regex("^/students/\\d+$")) -> {
                val id = path.removePrefix("/students/").toLong()
                handleGetStudent(id, resp)
            }
        }
    }

    private fun handleGetStudents(resp: HttpServletResponse) {
        val json = students.joinToString(",", "[", "]") { it.toJson() }
        resp.writer.write(json)
    }

    private fun handleGetStudent(id: Long, resp: HttpServletResponse) {
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

```
data class Student(val id: Long, val name: String, val number: Int) {
    fun toJson() = """{"id": $id, "name": "$name", "number": $number}"""
}
```

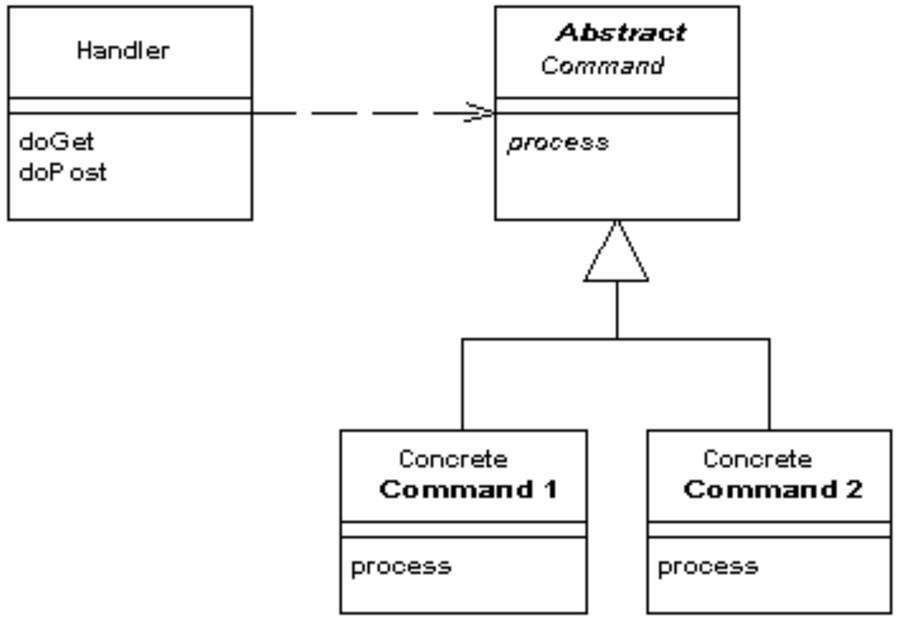
All handlers in the same component (PagesServlet)

This will not scale well

© Pedro Alves 2026

REST server implementation

Front Controller Pattern



https://martinfowler.com/eaaCatalog/frontController.html

© Pedro Alves 2026

REST server implementation

Servlet (Front Controller)

```
class PagesServlet : HttpServlet() {

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        val command = when {
            path == “/students" -> GetStudentsCommand(req, resp)
            path.matches(Regex("^/students/\\d+$")) -> GetStudentCommand(req, resp)
            else -> return
        }
        command.process()
    }
}
```

```
class GetStudentsCommand(req: HttpServletRequest,
                         resp: HttpServletResponse)
                         : FrontCommand(req, resp) {

    override fun process() {
        val json = students.joinToString(",", "[", "]")
                                { it.toJson() }
        resp.writer.write(json)
    }
}
```

implements

```
abstract class FrontCommand(val req: HttpServletRequest,
                            val resp: HttpServletResponse) {
    abstract fun process()
}
```

implements

```
class GetStudentCommand(req: HttpServletRequest,
                        resp: HttpServletResponse)
                        : FrontCommand(req, resp) {

    override fun process() {
        val id = req.pathInfo.removePrefix("/").toLong()
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

Front Controller Pattern

© Pedro Alves 2026

REST server implementation

Front Controller Pattern

The Front Controller Pattern (also known as Router), exists in all web frameworks

Example in Django (python):

urls.py

```
from django.urls import path

from .views import GetStudentsView, GetStudentView


urlpatterns = [

      path("students/", GetStudentsView.as_view()),

      path("students/<int:id>/", GetStudentView.as_view()),

]
```

© Pedro Alves 2026

REST server implementation

Servlet (Front Controller)

```
class PagesServlet : HttpServlet() {

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        val command = when {
            path == “/students" -> GetStudentsCommand(req, resp)
            path.matches(Regex("^/students/\\d+$")) -> GetStudentCommand(req, resp)
            else -> return
        }
        command.process()
    }
}
```

```
data class Student(val id: Long, val name: String, val number: Int) {
    fun toJson() = """{"id": $id, "name": "$name", "number": $number}"""
}
```

```
abstract class FrontCommand(val req: HttpServletRequest,
                            val resp: HttpServletResponse) {
    abstract fun process()
}
```

Problems with this implementation?

```
class GetStudentsCommand(req: HttpServletRequest,
                         resp: HttpServletResponse)
                        : FrontCommand(req, resp) {

    override fun process() {
        val json = students.joinToString(",", "[", "]")
                                { it.toJson() }
        resp.writer.write(json)
    }
}
```

```
class GetStudentCommand(req: HttpServletRequest,
                        resp: HttpServletResponse)
                        : FrontCommand(req, resp) {

    override fun process() {
        val id = req.pathInfo.removePrefix("/").toLong()
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

© Pedro Alves 2026

REST server implementation

Servlet (Front Controller)

```
class PagesServlet : HttpServlet() {

    override fun doGet(req: HttpServletRequest, resp: HttpServletResponse) {
        val path = req.requestURI.removePrefix(req.contextPath)
        resp.contentType = "application/json"

        val command = when {
            path == “/students" -> GetStudentsCommand(req, resp)
            path.matches(Regex("^/students/\\d+$")) -> GetStudentCommand(req, resp)
            else -> return
        }
        command.process()
    }
}
```

```
data class Student(val id: Long, va
    fun toJson() = """{"id": $id, "
}
```

```
abstract class FrontCommand(val req: HttpServletRequest,
                            val resp: HttpServletResponse) {
    abstract fun process()
}
```

- Parsing URLs
- Handling POST, PUT, …
- Producing JSON responses
- Handling errors (404, …)
- …

```
class GetStudentsCommand(req: HttpServletRequest,
                         resp: HttpServletResponse)
                        : FrontCommand(req, resp) {

    de fun process() {
    l json = students.joinToString(",", "[", "]")
                                { it.toJson() }
    sp.writer.write(json)
```

```
    tudentCommand(req: HttpServletRequest,
                  resp: HttpServletResponse)
                  : FrontCommand(req, resp) {

    override fun process() {
        val id = req.pathInfo.removePrefix("/").toLong()
        val student = students.find { it.id == id }
        if (student != null) {
            resp.writer.write(student.toJson())
        }
    }
}
```

© Pedro Alves 2026

GET /students

GET /students/{id}

REST server implementation

Spring MVC

```
data class Student(val id: Long, val name: String, val number: Int) {
    fun toJson() = """{"id": $id, "name": "$name", "number": $number}"""
}
```

No longer needed, Spring handles object serialization into JSON

```
@RestController
@RequestMapping("/students")
class StudentController {

    val students = listOf(
        Student(1, "Alice", 12345),
        Student(2, "Bob", 67890)
    )

    @GetMapping
    fun getStudents() = students

    @GetMapping("/{id}")
    fun getStudent(@PathVariable id: Long) = students.find { it.id == id }
}
```

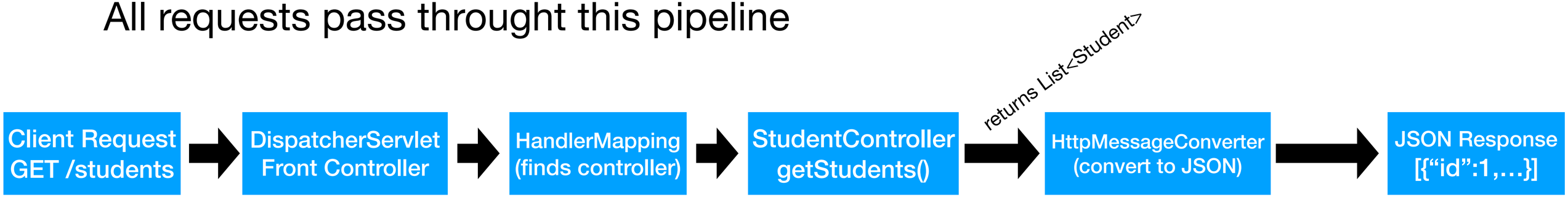
Routing is done through `@RequestMapping`, `@GetMapping`, etc., annotations

© Pedro Alves 2026

REST server implementation

Spring MVC

All requests pass throught this pipeline



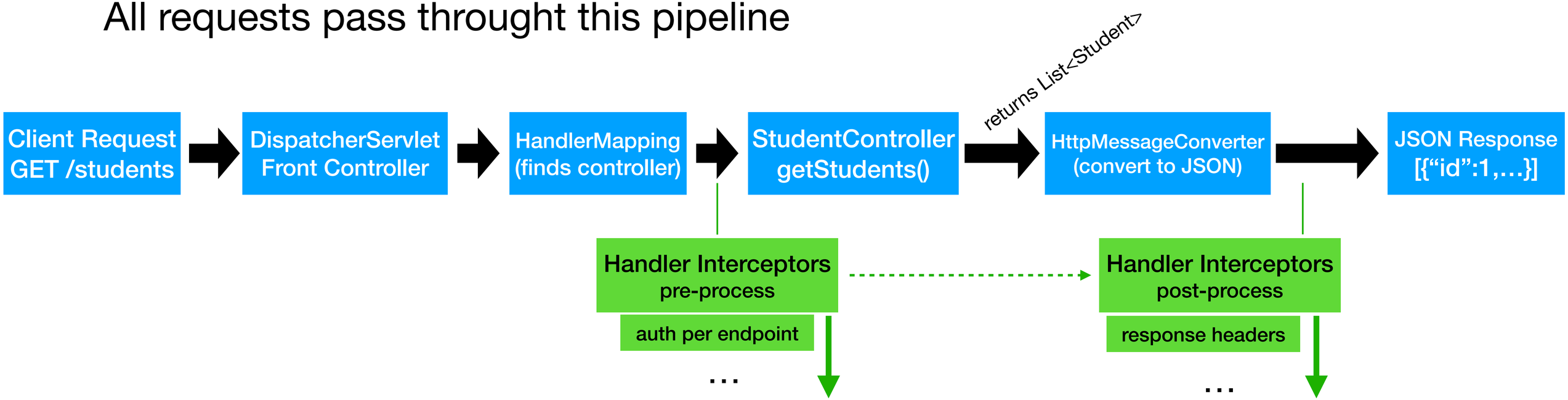
...

© Pedro Alves 2026

REST server implementation

Spring MVC

All requests pass throught this pipeline

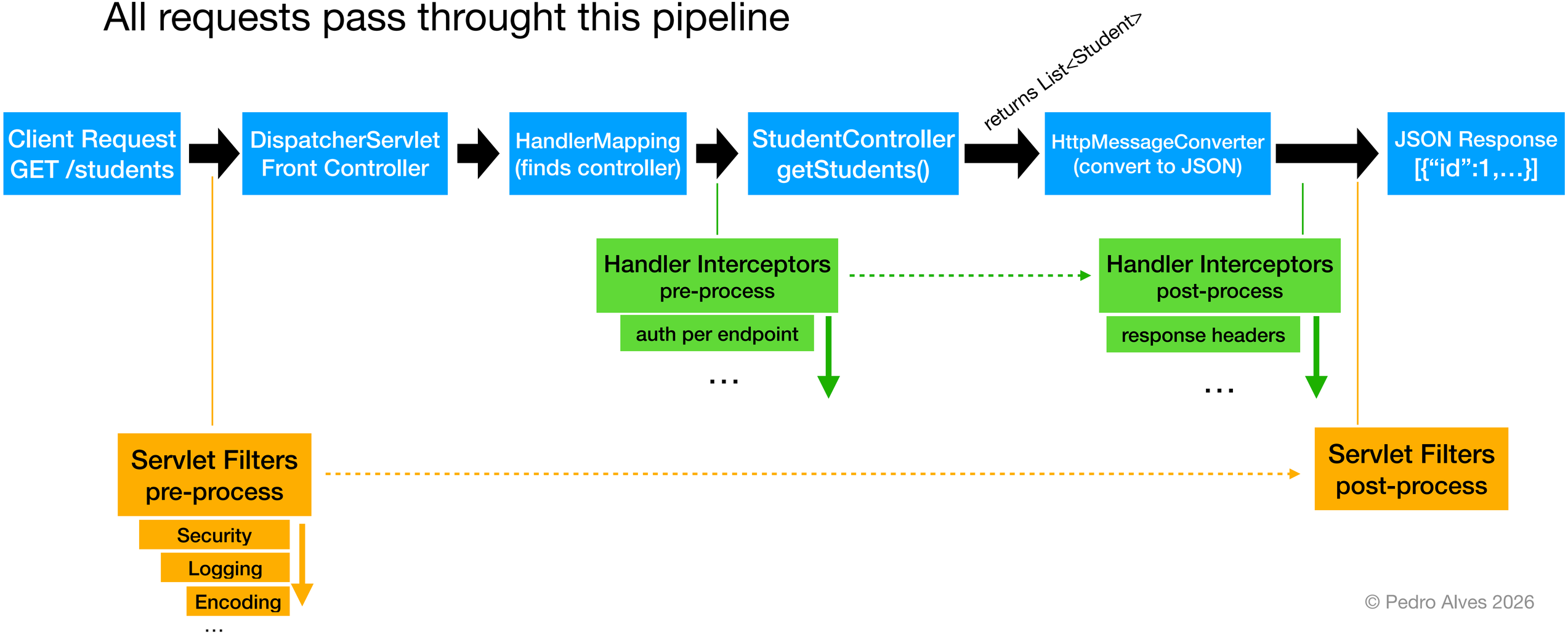


© Pedro Alves 2026

REST server implementation

Spring MVC

All requests pass throught this pipeline

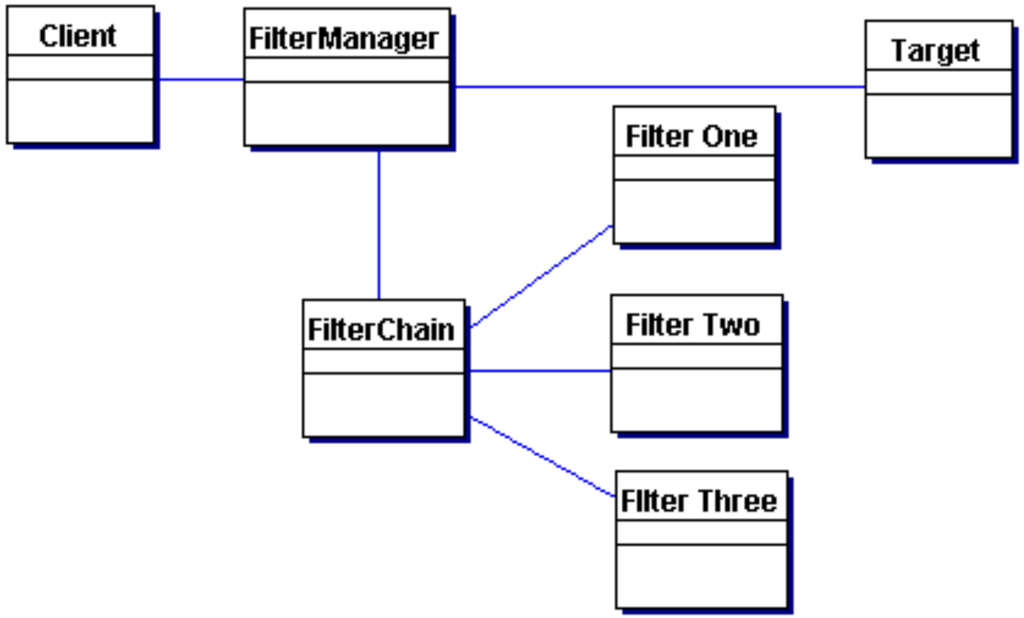


© Pedro Alves 2026

REST server implementation

Intercepting Filter Pattern

- filters intercept incoming requests and outgoing responses, allowing pre-processing and post-processing
- possible to add/remove filters without touching existing code
- used for common non-functional requirements (security, auditing, logging, …)



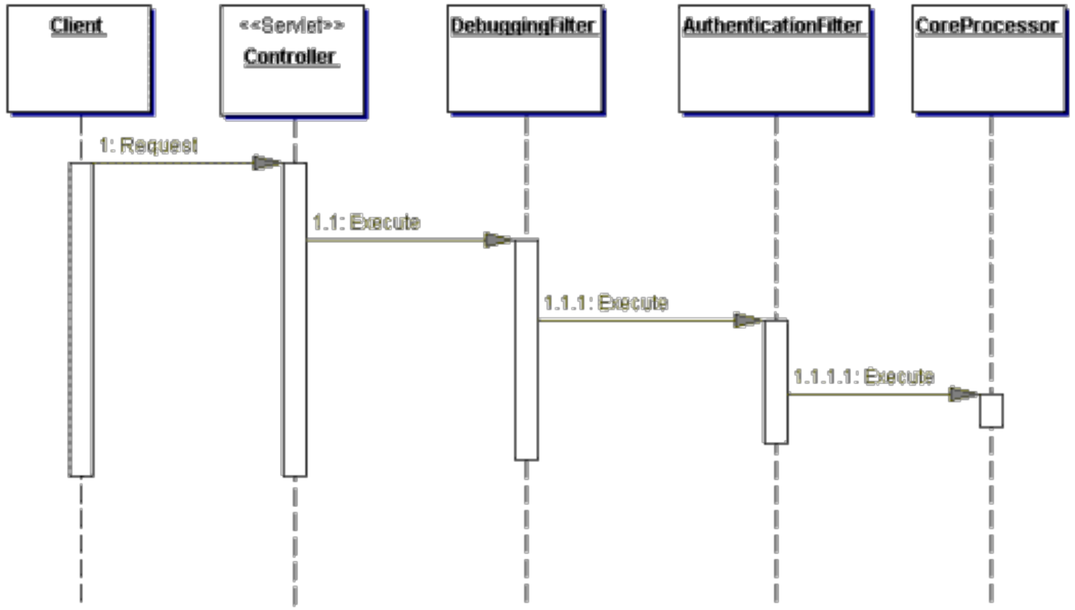
https://www.oracle.com/java/technologies/intercepting-filter.html

© Pedro Alves 2026

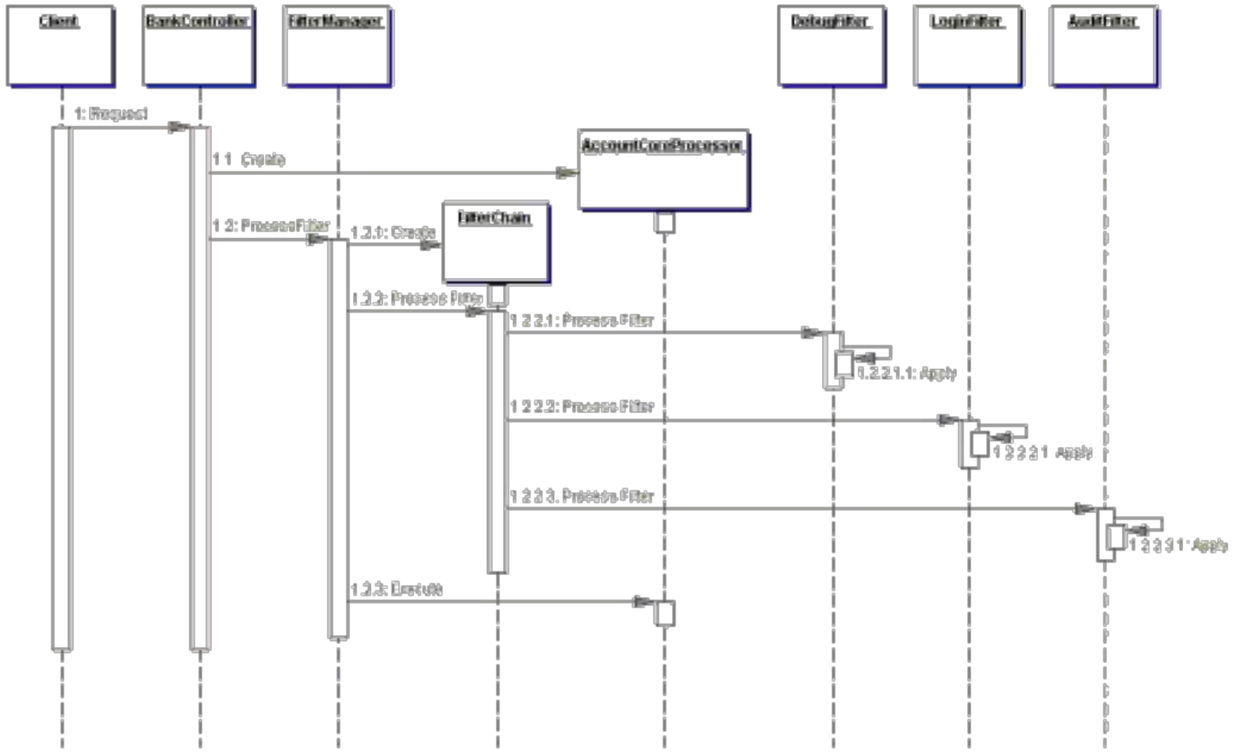
REST server implementation

Intercepting Filter Pattern

Decorated (nested) implementation



Non-Decorated implementation



© Pedro Alves 2026

REST server implementation

Intercepting Filter Pattern

Decorated (nested) implementation

```
Filter chain = new DebugFilter(
                   new LoginFilter(
                     new AuditFilter(
                       targetServlet)));



class AuditFilter implements Filter {
    private final Filter next;

    AuditFilter(Filter next) { this.next = next; }

    public void execute( … ) {
          // no pre-processing in this case
          next.execute(request, response);
          // post-processing: audit log
    }
}

// other filters…
```

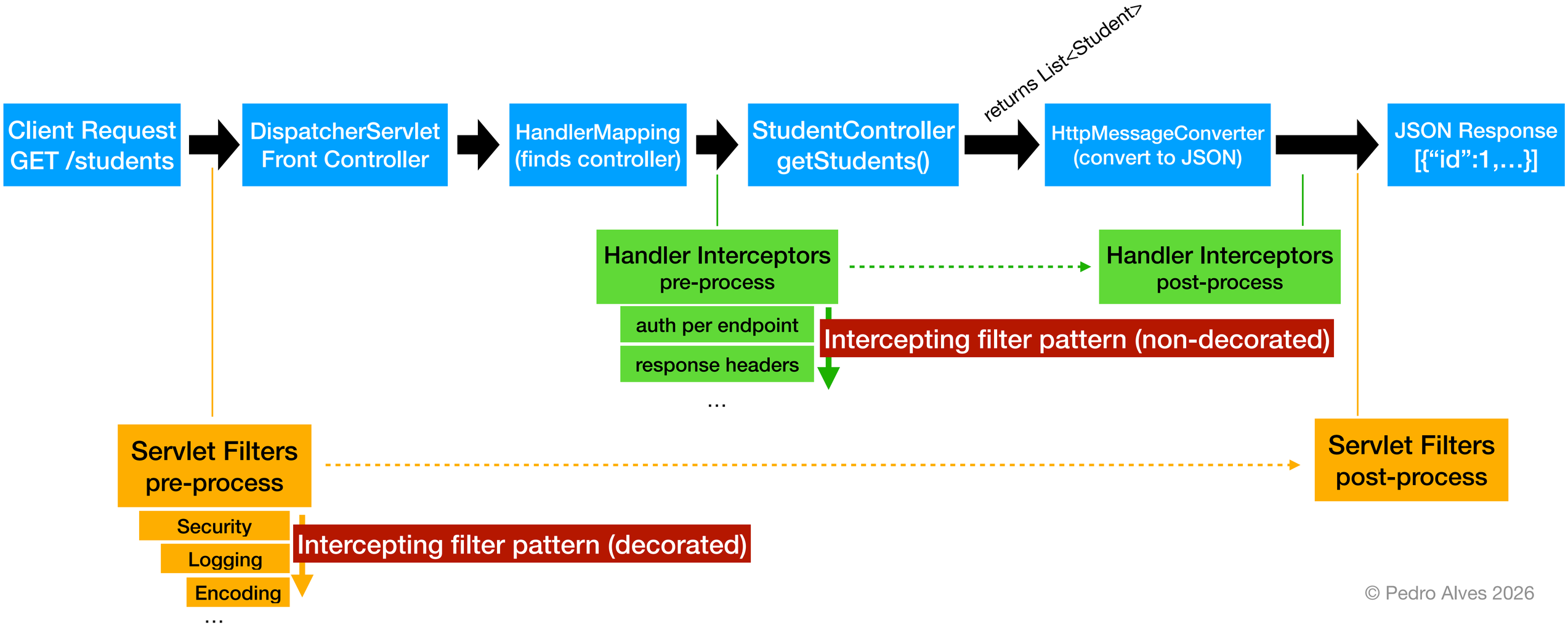
Non-Decorated implementation

```
public FilterChain()  {
    addFilter(new DebugFilter());
    addFilter(new LoginFilter());
    addFilter(new AuditFilter());
}


public void processFilter( … ) {
    Filter filter;

    // apply filters
    Iterator filters = myFilters.iterator();
    while (filters.hasNext())
    {
      filter = (Filter)filters.next();
      // pass request & response through various
      // filters
      filter.execute(request, response);
    }
  }
```

© Pedro Alves 2026

REST server implementation

Spring MVC



© Pedro Alves 2026

REST server implementation

Spring MVC

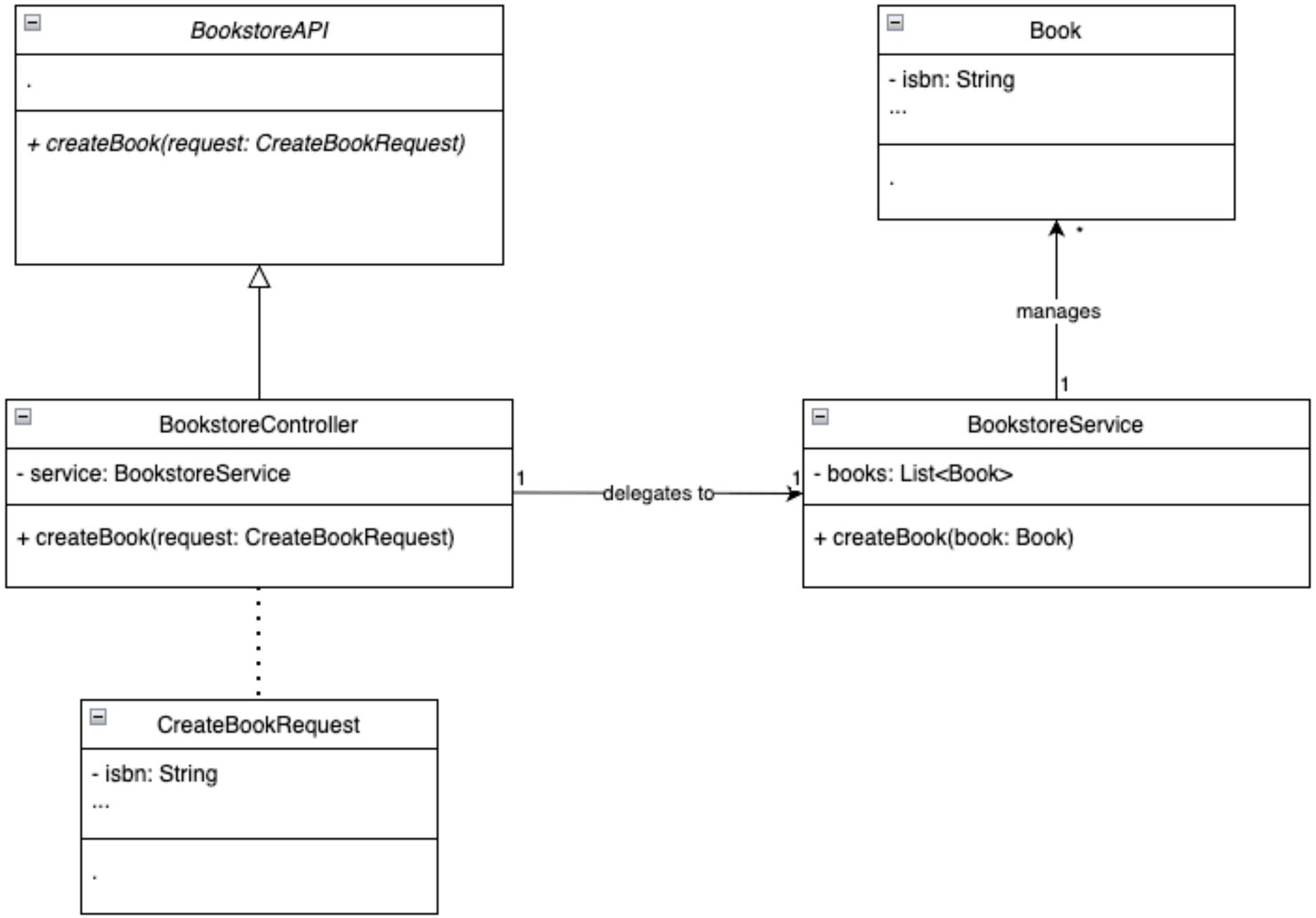
More info: https://docs.spring.io/spring-framework/reference/web/webmvc.html

© Pedro Alves 2026

REST server implementation

Class diagram

POST /books




```
bookstore
+—controller
  +—dto
    + CreateBookRequest
  + BookstoreAPI
  + BookstoreController
+—domain
  + Book
+—service
  + BookstoreService
```

© Pedro Alves 2026

REST server implementation

BookstoreAPI

```
interface BookstoreAPI {

    @Operation(summary = "Create a new book", operationId = "createBook")
    @ApiResponses(
        ApiResponse(responseCode = "201", description = "Book created",
            headers = [Header(name = "Location", description = "URI of the created book",
             schema = Schema(type = "string", format = "uri"))],
            content = [Content(schema = Schema(hidden = true))]),
        ApiResponse(responseCode = "400", description = "Validation error",
            content = [Content(schema = Schema(implementation = ErrorResponse::class))]),
        ApiResponse(responseCode = "409", description = "Book with this ISBN already exists",
            content = [Content(schema = Schema(implementation = ErrorResponse::class))])
    )
    @RequestMapping(
        value = ["/books"],
        consumes = ["application/json"],
        method = [RequestMethod.POST]
    )
    fun createBook(@Valid @RequestBody request: CreateBookRequest): ResponseEntity<Unit>


}
```

© Pedro Alves 2026

REST server implementation

BookstoreController

```
@RestController
class BookstoreController(private val service: BookstoreService) : BookstoreAPI {

   override fun createBook(request: CreateBookRequest): ResponseEntity<Unit> {

       val book = service.createBook(request.toBook())

       val location = ServletUriComponentsBuilder.fromCurrentRequest()
           .path("/{isbn}")
           .buildAndExpand(book.isbn)
           .toUri()

       return ResponseEntity.created(location).build()
   }

}
```

© Pedro Alves 2026

REST server implementation

CreateBookRequest

```
{
  "isbn": "9780134685991",
  "title": "Effective Java",
  "author": "Joshua Bloch",
  "price": 45.99,
  "image": "https://example.com/image.jpg"
}
```

```
data class CreateBookRequest(
    val isbn: String,
    val title: String,
    val author: String,
    val price: Double,
    val image: String,
)
```

Spring automatically unmarshalls JSON requests into classes

© Pedro Alves 2026

REST server implementation

CreateBookRequest

```
data class CreateBookRequest(

    @field:Schema(description = "Unique ISBN identifier", example = "9780134685991")
    val isbn: String,

    @field:Schema(description = "Title of the book", example = "Effective Java")
    val title: String,

    @field:Schema(description = "Author of the book", example = "Joshua Bloch")
    val author: String,

    @field:Schema(description = "Price of the book, must be greater than zero", example = "29.99")
    val price: Double,

    @field:Schema(description = "URL to the book cover image", example = "https://example.com/cover.jpg")
    val image: String,
)
```

These annotations will be shown in the OpenAPI spec

© Pedro Alves 2026

REST server implementation

CreateBookRequest

```
data class CreateBookRequest(
    @field:NotBlank
    @field:Schema(…)
    val isbn: String,

    @field:NotBlank
    @field:Size(min = 1, max = 120)
    @field:Schema(…)
    val title: String,

    @field:NotBlank
    @field:Size(min = 1, max = 80)
    @field:Schema(…)
    val author: String,

    @field:Positive
    @field:Schema(…)
    val price: Double,

    @field:NotBlank
    @field:URL
    @field:Schema(…)
    val image: String,
)
```

Using the validation library, Spring can validate the request fields

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

If the validation fails, a 400 (Bad Request) is returned along with json explaining the error

© Pedro Alves 2026

REST server implementation

DTO vs Domain

DTO (Data Transfer Object)

```
data class CreateBookRequest(
    @field:NotBlank
    @field:Schema(…)
    val isbn: String,

    @field:NotBlank
    @field:Size(min = 1, max = 120)
    @field:Schema(…)
    val title: String,

    …
)
```

encapsulate the serialization mechanism for transferring data over the wire

https://martinfowler.com/eaaCatalog/dataTransferObject.html

Domain

```
class Book(

    val isbn: String,

    val title: String,
    …
)
```

represents a business entity, encapsulating data and behaviour

https://martinfowler.com/eaaCatalog/domainModel.html

© Pedro Alves 2026

REST server implementation

DTO vs Domain

DTO (Data Transfer Object)

```
data class CreateBookRequest(
    @field:NotBlank
    @field:Schema(…)
    val isbn: String,

    @field:NotBlank
    @field:Size(min = 1, max = 120)
    @field:Schema(…)
    val title: String,

    …
)
```

Domain

```
class Book(

    val isbn: String,

    val title: String,

    var lastUpdated: Date,
    …
)
{
    fun calculateDiscount(): Int {
        ...
    }
}
```

this field doesn't make sense in the Request object

behaviour is allowed in domain objects but not DTOs

Common mistake 1:

Use domain objects as DTOs (tight coupling between requests and business entities)

Common mistake 2:

Think that there must be a direct relationship between both

© Pedro Alves 2026

REST server implementation

DTO vs Domain

DTOs should also be used for responses

```
data class GetBookResponse(

    @field:Schema(…)
    val isbn: String,

    @field:Schema(…)
    val title: String,

    …
)
```

Note: Book -> GetBookResponse must be done manually (and that’s good)

© Pedro Alves 2026

REST server implementation

DTO vs Domain

The transformation of DTOs into Domain objects and vice-versa must be done manually (and that's good)

```
data class CreateBookRequest(
    @field:NotBlank
    @field:Schema(…)
    val isbn: String,

    @field:NotBlank
    @field:Size(min = 1, max = 120)
    @field:Schema(…)
    val title: String,

    …
)
```

```
class Book(

    val isbn: String,

    val title: String,
    …
)
```

```
val book = Book(
    isbn = request.isbn,
    title = request.title,
    author = request.author,
    price = request.price,
    image = request.image,
)
```

inside the controller

© Pedro Alves 2026

REST server implementation

DTO vs Domain

The transformation of DTOs into Domain objects and vice-versa must be done manually (and that's good)

```
data class CreateBookRequest(
    @field:NotBlank
    @field:Schema(…)
    val isbn: String,

    @field:NotBlank
    @field:Size(min = 1, max = 120)
    @field:Schema(…)
    val title: String,

    …
)

{
    fun toBook() = Book(
        isbn = isbn,
        title = title,
        author = author,
        price = price,
        image = image,
    )
}
```

```
val book = request.toBook()
```

```
class Book(

    val isbn: String,

    val title: String,
    …
)
```

© Pedro Alves 2026

REST server implementation

Controller -> Service

```
@Service
class BookstoreService {

    fun createBook(book: Book) {
        ...
    }

}
```

Dependency Injection

```
@RestController
class BookstoreController(private val service: BookstoreService) : BookstoreAPI {

    override fun createBook(request: CreateBookRequest): ResponseEntity<Unit> {

        val book = service.createBook(request.toBook())

        …
    }

}
```

Business logic outside the controller

© Pedro Alves 2026

Tratamento de erros

REST

- Resource Based
- Representation
- Uniform Interface
- Stateless
- Cacheable
- Client-Server
- Layered System
- Code on Demand (optional, not talking about it)

Errors should handled the same way, regardless of the resource

© Pedro Alves 2026

Tratamento de erros

Errors should handled the same way, regardless of the resource

BookNotFoundException

```
GET /books/{isbn}
PUT /books/{isbn}
```

ReviewNotFoundException

```
GET ...
PUT ...
```

```
HTTP/1.1 404
Content-Language: en
Content-Type: application/json

{
  "error": "NOT_FOUND",
  "message": “Some explanation…”
}
```

© Pedro Alves 2026

REST server implementation

Tratamento de erros

GET /books/{isbn}

```
override fun getBook(isbn: String, locale: Locale): ResponseEntity<*> {
    return try {
        ResponseEntity.ok(BookResponse.fromBook(service.getBook(isbn)))
    } catch (ex: BookNotFoundException) {
        ResponseEntity.status(404).body(ErrorResponse("NOT_FOUND", message))
    }
}
```

Error handling is duplicated

PUT /books/{isbn}

```
override fun replaceBook(isbn: String, request: CreateBookRequest): ResponseEntity<*> {
    return try {
        ResponseEntity.ok(BookResponse.fromBook(service.replaceBook(isbn, request.toBook())))
    } catch (ex: BookNotFoundException) {
        ResponseEntity.status(404).body(ErrorResponse("NOT_FOUND", message))
    }
```

© Pedro Alves 2026

REST server implementation

Tratamento de erros

```
@RestController
class BookstoreController(private val service: BookstoreService) : BookstoreAPI {

   override fun getBook(isbn: String): ResponseEntity<BookResponse> =
       ResponseEntity.ok(BookResponse.fromBook(service.getBook(isbn)))

}
```

No error handling in controllers

© Pedro Alves 2026

REST server implementation

Tratamento de erros

```
@RestController
class BookstoreController(private val service: BookstoreService) : BookstoreAPI {

   override fun getBook(isbn: String): ResponseEntity<BookResponse> =
       ResponseEntity.ok(BookResponse.fromBook(service.getBook(isbn)))

}
```

No error handling in controllers

```
@RestControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(BookNotFoundException::class)
    fun handleBookNotFound(ex: BookNotFoundException, locale: Locale):
                            ResponseEntity<ErrorResponse> {
       return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ErrorResponse("NOT_FOUND", “some message”))
    }
}
```

Error handling concentrated in GlobalExceptionHandler

© Pedro Alves 2026

HTTP Headers

▼ Request Headers

:authority	www.wikipedia.org
:method	GET
:path	/
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=(exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	pt-PT,pt;q=0.9,en-US;q=0.8,en;q=0.7,smn-FI;q=0.6
Cookie	WMF-Last-Access-Global=22-Jan-2026; WMF-Uni
Priority	u=0, i
Referer	https://duckduckgo.com/
Sec-Ch-Ua	"Not(A:Brand";v="8", "Chromium";v="144", "Googl
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"macOS"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	cross-site
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Ap

▼ Response Headers

Accept-Ranges	bytes
Age	40160
Cache-Control	s-maxage=86400, must-revalidate, max-age=3600
Content-Encoding	gzip
Content-Length	48196
Content-Type	text/html
Date	Wed, 18 Feb 2026 06:08:54 GMT
Etag	W/"2f734-6494d2185f300"
Last-Modified	Mon, 26 Jan 2026 16:38:04 GMT
Nel	{ "report_to": "wm_nel", "max_age": 604800, "failure_fraction": 0.0
Report-To	{ "group": "wm_nel", "max_age": 604800, "endpoints": [{ "url": "htt stream=w3c.reportingapi.network_error&schema_uri=/w3c/reportin
Server	ATS/9.2.11
Server-Timing	cache;desc="hit-front", host;desc="cp6011"

© Pedro Alves 2026

REST server implementation

Headers HTTP

```
@RestController
class BookstoreController(
    private val service: BookstoreService,
    private val httpRequest: HttpServletRequest
    ) : BookstoreAPI {

    override fun createBook(request: CreateBookRequest): ResponseEntity<Unit> {

        val requestId = httpRequest.getHeader("X-Request-Id")
        println("Incoming requestId: $requestId")

        val book = service.createBook(request.toBook())
        val location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{isbn}")
            .buildAndExpand(book.isbn)
            .toUri()
        return ResponseEntity
            .created(location)
            .header("X-Request-Id", requestId ?: "generated-id-123")
            .build()
    }
}
```

Spring injects a low-level access to the HttpRequest

get header from low level httpRequest, don’t confuse wirh request (payload)

set header in response

© Pedro Alves 2026

REST server implementation

Headers HTTP

Example: Language negotiation (the API must be multi-lingual)

```
GET /en/books/{isbn}
GET /pt/books/{isbn}
```

❌ This is not good REST

```
GET /books/{isbn}
Accept-Language: en
GET /books/{isbn}
Accept-Language: pt
```

✅ This is good REST

© Pedro Alves 2026

REST server implementation

Testing the API in Intellij

api.http

Run with: No Environment

```http
### US1 – List all books
GET http://localhost:8080/books
Accept: application/json

### US2 – Create a book
POST http://localhost:8080/books
Content-Type: application/json

{
  "isbn": "9780134685991",
  "title": "Effective Java",
  "author": "Joshua Bloch",
  "price": 19.99,
  "image": "https://example.com/effective-java.jpg"
}
```

https://www.jetbrains.com/help/idea/2025.3/http-client-in-product-code-editor.html

© Pedro Alves 2026

Exercise

Spot the errors

```
1. @RestController
2. @RequestMapping("/api")
3. class StudentController(val studentService: StudentService) {
4.
5.     private var students: MutableList<Student>? = null
6.
7.     @GetMapping("/students")
8.     fun getStudents(): ResponseEntity<List<Student>?> {
9.         return ResponseEntity.ok(students)
10.     }
11.
12.     @PostMapping("/students/delete")
13.     fun deleteStudent(@RequestParam id: Long): ResponseEntity<String> {
14.         students!!.removeIf { it.id == id }
15.         return ResponseEntity.ok("deleted")
16.     }
17.
18.     @PutMapping("/students/{id}")
19.     fun replaceStudent(@PathVariable id: Long,
20.                        @RequestBody student: Student): ResponseEntity<Student> {
21.        students?.removeIf { it.id == id }
22.        students?.add(student)
23.        return ResponseEntity.status(HttpStatus.CREATED).body(student)
24.    }
25.
26.    @GetMapping("/students/search")
27.    fun search(@RequestBody name: String): ResponseEntity<List<Student>> {
28.        return ResponseEntity.ok(students!!.filter { it.name.contains(name) })
29.    }
30.}
```

© Pedro Alves 2026

Resolution

```
1. @RestController
2. @RequestMapping("/api")
3. class StudentController(val studentService: StudentService) {
4.
5.     private var students: MutableList<Student>? = null
6.
7.     @GetMapping("/students")
8.     fun getStudents(): ResponseEntity<List<Student>?> {
9.         return ResponseEntity.ok(students)
10.     }
11.
12.     @PostMapping("/students/delete")
13.     fun deleteStudent(@RequestParam id: Long): ResponseEntity<String> {
14.         students!!.removeIf { it.id == id }
15.         return ResponseEntity.ok("deleted")
16.     }
17.
18.     @PutMapping("/students/{id}")
19.     fun replaceStudent(@PathVariable id: Long,
20.                        @RequestBody student: Student): ResponseEntity<Student> {
21.         students?.removeIf { it.id == id }
22.         students?.add(student)
23.         return ResponseEntity.status(HttpStatus.CREATED).body(student)
24.     }
25.
26.     @GetMapping("/students/search")
27.     fun search(@RequestBody name: String): ResponseEntity<List<Student>> {
28.         return ResponseEntity.ok(students!!.filter { it.name.contains(name) })
29.     }
30.}
```

<< doesn't implement StudentAPI

<< this variable belongs to studentService; should be initialized with empty list

<< List<Student> shouldn't be nullable

<< DeleteMapping("/students/{id}")

<< This will crash on null

<< Shouldn't returns Strings; actually DELETE should return 204 (No Content)

<< Should only be CREATED if it didn't exist

<< Not a RESTful endpoint

General error: business logic should be done in StudentService, not the controller

© Pedro Alves 2026